

Coding for Artificial Intelligence

Advanced Function Signatures

Problem Set 7

Project: The Autonomous Payload Packet Parser

Objective

The objective of this practical is to create a telemetry parser that works without changing the original packet. The program uses positional-only and keyword-only parameters and creates a separate sensor list using slicing and list comprehension.

Function Signature

The required function signature is:

```
def parse_payload(raw_packet: list, delimiter: str, /, *,
                  correction_offset: int = 0) -> tuple:
```

The parameters before / are positional-only. The parameter after * is keyword-only.

Source Code

```
def parse_payload(raw_packet: list, delimiter: str, /, *,
                  correction_offset: int = 0) -> tuple:
    status_tokens = raw_packet[2].split(delimiter)

    corrected_sensors = [
        value if "ERROR" in status_tokens else value + correction_offset
        for value in raw_packet[1][:]
    ]

    return raw_packet[0], corrected_sensors

raw_packet1 = [501, [12.5, 13.0, 11.8], "NOMINAL|CALIBRATED"]
result1 = parse_payload(raw_packet1, "|", correction_offset=2)

print("Test_1->", result1[0], result1[1])

assert raw_packet1[1] == [12.5, 13.0, 11.8]
assert id(result1[1]) != id(raw_packet1[1])

print("Test_1-side-effect_assertions_passed.")
```

```

raw_packet2 = [502, [9.0, 8.5], "ERROR|SENSOR_FAULT"]
result2 = parse_payload(raw_packet2, "|", correction_offset=5)

print("Test_2->", result2[0], result2[1])

assert raw_packet2[1] == [9.0, 8.5]
assert id(result2[1]) != id(raw_packet2[1])

print("Test_2_side-effect_assertions_passed.")

raw_packet3 = [503, [1.0, 2.0], "NOMINAL"]
result3 = parse_payload(raw_packet3, "|")

print("Test_3(default_offset)->", result3[0], result3[1])

try:
    parse_payload(raw_packet1, delimiter="|")
except TypeError as e:
    print("Test_4_passed.TypeError_raised_as_expected:", e)

try:
    parse_payload(raw_packet1, "|", 2)
except TypeError as e:
    print("Test_5_passed.TypeError_raised_as_expected:", e)

raw_packet4 = [504, [], "NOMINAL"]
result4 = parse_payload(raw_packet4, "|", correction_offset=5)

print("Test_6->", result4[0], result4[1])

assert result4[1] == []
assert id(result4[1]) != id(raw_packet4[1])

```

Output

The program produced the following output:

```

Test 1 -> 501 [14.5, 15.0, 13.8]
Test 1 side-effect assertions passed.

Test 2 -> 502 [9.0, 8.5]
Test 2 side-effect assertions passed.

Test 3 (default offset) -> 503 [1.0, 2.0]

```

```
Test 4 passed. TypeError raised as expected:
parse_payload() got some positional-only arguments passed as keyword arguments

Test 5 passed. TypeError raised as expected:
parse_payload() takes 2 positional arguments but 3 were given

Test 6 -> 504 []
```

Explanation

For Test 1, the packet does not contain the **ERROR** status. Therefore, the correction offset of 2 is added to every sensor value.

For Test 2, the status contains **ERROR**. Therefore, the correction offset is not applied and the original sensor values are returned in a new list.

Test 3 checks the default value of the correction offset.

Tests 4 and 5 check the function signature. Passing positional-only parameters by keyword and passing the keyword-only parameter positionally both produce a **TypeError**.

The assertions also confirm that the original sensor list is unchanged and that the returned list is a different object.

Conclusion

The practical successfully implements the Autonomous Payload Packet Parser. The program uses positional-only and keyword-only parameters, handles the **ERROR** status correctly, creates an independent sensor list, and verifies that the original packet is not modified.